

Python PBL 03

« Поиск угроз безопасности с помощью наблюдения
за папкой »



SK Shieldus

Rookies, 30-й набор (личный номер 14)

Ким Джэ-Ук

Решение PBL 03

```
# работа с системой, а именно пути, папки и списки файлов
import os
# регулярные выражения для поиска по шаблону
import re
# ожидание между проверками
import time
# системные функции
import sys

# папка, за которой ведётся наблюдение
DIR = "./monitor_directory"
# набор расширений, при виде которых выводится предупреждение
EXTS = (".js", ".class", ".py")

# список пар из названия типа и объекта регулярного выражения,
# если строка подходит под шаблон, значит она найдена
RULES = [
    # все строки, которые начинаются с # или //, то есть сюда
    # попадают не только комментарии Python, но и комментарии
    # JavaScript или Java
    # при этом строка считается комментарием, только если # или //
    # стоят в самом её начале, не считая пробелов, потому что
    # иначе по ошибке находился бы адрес, начинающийся с http://
    ("comments", re.compile(r"^\s*(#|//).*")),
    # самый обычный вид почтового адреса, то есть hello@domainname.com
    ("email", re.compile(r"\b[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\b")),
    # ключевые слова языка запросов SQL
    ("sql", re.compile(r"ALTER|CREATE|DELETE|DROP|EXEC|GRANT|INSERT|REPLACE|REVOKE|"
        r"SELECT|TRUNCATE|UPDATE", re.IGNORECASE)),
]
# параметр re.IGNORECASE добавлен, чтобы находить слова независимо
# от регистра букв, ведь так ломается приём взломщиков, которые
# смешивают заглавные и строчные буквы ради обхода проверки

# функция, которая собирает текущий список файлов папки в множество
def snapshot(path):
    # возможны ошибки с правами или с путём, поэтому берём try
    try:
        # возвращаем множество имён файлов и папок внутри папки
        return set(os.listdir(path))
    # ловим любую ошибку и возвращаем пустое множество
    except Exception:
        # при неудаче возвращаем пустое множество,
        # чтобы наблюдение не прервалось
```

```

    return set()

# функция, которая открывает файл, проверяет его построчно
# и собирает все находки
def inspect(path):
    # список находок в виде номера строки, содержимого и типа
    hits = []
    # открываем файл как текст в кодировке utf-8, он закроется сам
    with open(path, "r", encoding="utf-8") as f:
        # читаем файл построчно и сразу берём номер строки с единицы
        for ln, s in enumerate(f, 1):
            # убираем только знак перевода строки в конце
            s2 = s.rstrip("\n")
            # перебираем каждую пару из типа и регулярного выражения
            for kind, rx in RULES:
                # если строка подходит под шаблон
                if rx.search(s2):
                    # записываем номер строки, содержимое и тип
                    hits.append((ln, s2.strip(), kind))
    # после проверки всего файла возвращаем список находок
    return hits

# один цикл наблюдения, а именно поиск новых файлов,
# их проверка и вывод результата
def step(path, seen):
    # берём текущий список файлов, при ошибке будет пустое множество
    now = snapshot(path)
    # новыми считаем только те файлы, которых не было в прошлом списке
    added = now - seen
    # если появился хотя бы один новый файл
    if added:
        print(f"[INFO] Добавлен новый файл {added}")
        # обрабатываем новые файлы по одному
        for name in added:
            # соединяем путь к папке и имя файла в полный путь
            full = os.path.join(path, name)

            # проверяем, оканчивается ли имя на подозрительное расширение
            if name.endswith(EXTS):
                print(f"[WARNING] Найден файл для проверки {name}")

            try:
                # проверяем содержимое файла и получаем список находок
                found = inspect(full)

                # если находки есть, то есть найден хотя бы один шаблон

```

```

    if found:
        print(f"[ALERT] В файле {name} есть содержимое для проверки")
        # достаём находки по одной и выводим их
        for ln, txt, kind in found:
            print(f" - тип {kind}, строка {ln}, содержимое {txt}")
        # ловим любую ошибку, включая сбой чтения
    except Exception as e:
        print(f"[ERROR] Сбой при чтении файла {name}, {e}")
    # возвращаем свежий список файлов этого цикла
    return now

# этот блок выполняется только при прямом запуске файла
if __name__ == "__main__":
    # проверяем, что папка для наблюдения действительно существует
    if not os.path.exists(DIR):
        print(f"[ERROR] Путь для наблюдения не найден {DIR}")
        # завершаем программу, дальше работать нельзя
        sys.exit()

    # сообщаем о начале наблюдения
    print(f"[INFO] Начинаем наблюдение за папкой {DIR}")
    # запоминаем список файлов на момент старта,
    # чтобы старые файлы не считались новыми
    seen = snapshot(DIR)

    # крутим бесконечный цикл, пока пользователь его не остановит
    try:
        # программа работает и проверяет папку снова и снова
        while True:
            # выполняем один цикл и обновляем список в seen
            seen = step(DIR, seen)
            # ждём одну секунду и проверяем заново
            time.sleep(1)
        # ошибка, которая возникает, когда пользователь просит остановиться
    except KeyboardInterrupt:
        print(f"[INFO] Наблюдение остановлено по запросу пользователя")

```

Создание тестовых файлов, подготовка и запуск наблюдения

По одному только коду выше трудно убедиться, что наблюдение действительно работает, поэтому мы сами создаём тестовые файлы один за другим и проверяем, работает ли код так, как было задумано.

```
# переходим в рабочую папку PBL3
cd PBL3
# создаём простой текстовый файл для проверки, внутри слово hello
echo "hello" > "./monitor_directory/new.txt"
# создаём файл с расширением .py, которое входит в список
# подозрительных, внутри строка print('hi')
echo "print('hi')" > "./monitor_directory/suspicious.py"
# создаём файл, внутри которого сразу несколько видов
# проверяемого содержимого, а именно комментариев, почта и запрос SQL
cat > "./monitor_directory/leak.txt" << 'EOF'
# comment line          # комментарий в стиле Python
// comment line 2       # комментарий в стиле JavaScript
test@example.com        # образец для поиска почты
SELECT * FROM users;    # образец с ключевым словом SELECT
SeLeCt                 # то же слово, но буквы разного регистра
EOF                    # конец ввода
```

Результаты проверки наблюдения

```
[INFO] Начинаем наблюдение за папкой ./monitor_directory
[INFO] Добавлен новый файл {'DS_Store'}
[ERROR] Сбой при чтении файла .DS_Store, 'utf-8' codec can't decode byte 0x80 in position 3131: invalid start byte
[INFO] Добавлен новый файл {'new.txt'}
[INFO] Добавлен новый файл {'suspicious.py'}
[WARNING] Найден файл для проверки suspicious.py
[INFO] Добавлен новый файл {'leak.txt'}
[ALERT] В файле leak.txt есть содержимое для проверки
- тип comments, строка 1, содержимое # comment line
- тип comments, строка 2, содержимое // comment line 2
- тип email, строка 3, содержимое test@example.com
- тип sql, строка 4, содержимое SELECT * FROM users;
- тип sql, строка 5, содержимое SeLeCt
```

По ходу выполнения приведённого выше кода на Python и по его результатам можно убедиться, что все цели, поставленные в задании, полностью достигнуты.

Углублённое толкование результатов

Если учитывать смысл задания, а именно «**ради безопасности системы наблюдать за появлением новых файлов в определённой папке и находить файлы и содержимое, способные стать угрозой безопасности**», то приведённое ниже **толкование результатов** тоже оказывается осмысленным.

- (1) **Файл, найденный самым первым**, а это **.DS_Store**, представляет собой системный файл, который в macOS создаётся сам собой. Изначально он не является объектом проверки безопасности, но из-за двоичного формата возникла ошибка декодирования UTF-8. Это показывает, что в наблюдение могут попадать лишние или необычные файлы в зависимости от операционной системы, и говорит о том, что на практике для таких системных файлов нужна отдельная обработка исключений.
- (2) **Найденный следом файл new.txt** содержит только простой текст, его расширение не входит в список подозрительных, и внутри нет ни одного проверяемого шаблона, а именно комментариев, почтовых адресов и запросов SQL, поэтому он оценивается как обычный файл без угроз безопасности. Это показывает, что система наблюдения не считает опасным любой новый файл подряд, а умеет отличать обычные файлы за счёт разбора содержимого.
- (3) **Файл suspicious.py с подозрительным расширением** был найден как исполняемый файл сценария с расширением .py, и поэтому появилось предупреждение. Хотя в самом содержимом файла дополнительных проверяемых шаблонов не нашлось, это говорит о том, что уже сам факт появления нового файла с исполняемым кодом позволяет отнести его к объектам, требующим проверки безопасности.
- (4) В **самом важном с точки зрения безопасности файле leak.txt** подряд были найдены комментарии (# и //), почтовый адрес (test@example.com) и ключевое слово SQL (SELECT, в том числе при смешении заглавных и строчных букв). Это значит, что одновременно есть и возможность утечки чувствительных сведений, и риск, связанный с изменением базы данных, и это же главный результат, который показывает, что данная система наблюдения не просто замечает появление файлов, а действительно находит угрозы безопасности по содержимому.

Этот документ подготовлен в формате **Quarto Markdown**(.qmd), который позволяет объединять код **Python** и текст документа в одном файле, и выведен в формате PDF.